

DM2026 – LAB1

Master Notebook Guide

LEARN THE DATA MINING CONCEPTS BEHIND IT

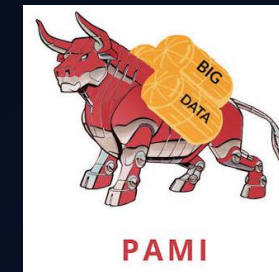
ISA5810 DATA MINING LAB 1

Table of Contents

- Python Libraries Overview
- Notebook Content Overview
- 1. Data Source
- 2. Data Preparation
- 3. Data Transformation
- 4. Data Mining using Pandas
- 5. Data Preprocessing
- 6. Data Exploration
- Recommendations

Python Libraries Overview

- **Data Wrangling & Math:**
 - **pandas:** Tabular data manipulation, missing value handling, querying
 - **numpy:** Fast vectorized array and matrix operations
- **Text Processing & Machine Learning:**
 - **nlTK:** Natural language toolkit for sentence and word tokenization
 - **scikit-learn:** Feature extraction (CountVectorizer), variance filtering, and cosine similarity
- **Pattern Mining & Manifold Learning:**
 - **PAMI:** Pattern Mining library used to run the following algorithms: FPGrowth, MaxFPGrowth, FAE, Association Rule Mining
 - **umap-learn:** Fast non-linear dimensionality reduction for cluster visualization
- **Plotting:**
 - **matplotlib & seaborn:** Static distribution plots, boxplots, and heatmaps
 - **plotly & kaleido:** Making interactive 2D and 3D scatter plots, and making them static to export.



Notebook Content Overview

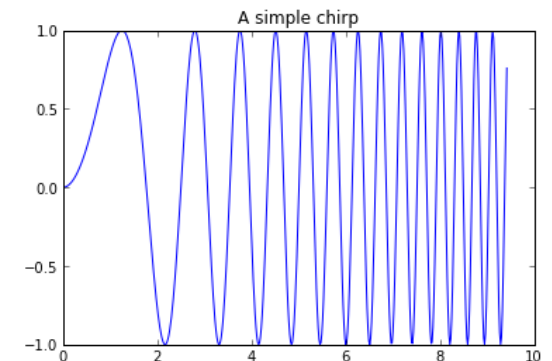
Jupyter & Helper Modules

- **Interactive Environment:**
 - Jupyter provides cell-by-cell execution, immediate visual feedback, and easy inspection of data shapes at each stage.
- **Modular Code Structure (helpers/ folder):**
 - Repeated logic is separated into clean, reusable Python scripts rather than cluttering the notebook cells.
 - Inspect these files to see how text cleaning, custom filtering, and plotting functions work behind the scenes.

Jupyter Notebook Example

```
In [3]: from matplotlib.pyplot import *  
x = linspace(0, 3*pi, 500)  
plot(x, sin(x**2))  
title("A simple chirp")
```

Out[3]: <matplotlib.text.Text at 0x315afd0>



Python Helpers Directory

```
▼ helpers  
  > __pycache__  
  + __init__.py  
  + data_mining_helpers.py  
  + data_persistence_helpers.py  
  + feature_filtering.py  
  + text_analysis.py  
  + text_filtering_helpers.py  
  + visualization_helpers.py
```

Notebook Content Overview

Take-home questions (notebook order):

- **Q1 Data Filtering & Domain Relevance** (Section 3.2)
- **Q2 Handling Missing data** (Section 4.1)
- **Q3 Data Sampling Risks** (Section 5.1)
- **Q4 Text Vectorization and Matrix Sparsity** (Section 5.3)
- **Q5 Feature Selection — Variance Thresholding** (Section 5.3.1)
- **Q6 Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)** (Section 5.3.2)
- **Q7 Frequent Pattern Mining — Algorithm & Parameter Dynamics** (Section 5.4.2)
- **Q8 Dimensionality Reduction — Motivation** (Section 5.5)
- **Q9 Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)** (Section 5.5)
- **Q10 Target Encoding — Integer vs. One-Hot Binarization** (Section 5.6)

1. Data Source

The 20 Newsgroups Dataset

- **Benchmark Corpus:**
 - ~20,000 forum posts across 20 distinct Usenet discussion groups.
 - Standard public benchmark for natural language text classification, clustering, and retrieval.
- **Raw Unstructured Data:**
 - Consists of messy email-style headers (From:, Subject:, Lines:), user quotes, and informal text bodies.
 - Raw text cannot be used directly by machine learning models; it must be parsed and represented numerically.

	text	category_name
0	From: sd345@city.ac.uk (Michael Collier) Subje...	comp.graphics
1	From: ani@ms.uky.edu (Aniruddha B. Deglurkar) ...	comp.graphics
2	From: djohnson@cs.ucsd.edu (Darin Johnson) Sub...	soc.religion.christian
3	From: s0612596@let.rug.nl (M.M. Zwart) Subject...	soc.religion.christian
4	From: stanly@grok11.columbiasc.ncr.com (stanly...	soc.religion.christian
5	From: vbv@lor.eeap.cwru.edu (Virgilio (Dean) B...	soc.religion.christian
6	From: jodfishe@silver.ucs.indiana.edu (Joseph ...	soc.religion.christian
7	From: aldridge@netcom.com (Jacquelin Aldridge)...	sci.med
8	From: geb@cs.pitt.edu (Gordon Banks) Subject: ...	sci.med
9	From: libman@hsc.usc.edu (Marlena Libman) Subj...	sci.med

2. Data Preparation

Loading and Inspecting Raw Text

- **Controlled Category Subset:**
 - Loading 4 selected topics: comp.graphics, sci.med, alt.atheism, and soc.religion.christian (2,257 total posts).
 - Balances technical and philosophical domains while keeping computation fast.
- **Reproducibility:**
 - `random_state=42` guarantees identical document shuffling across runs.
- **2.1 Finding data files on disk:**
 - Using Python's `os.listdir` and `os.path.isdir` to inspect local project files and output folders.

3. Data Transformation

Unstructured Text to DataFrames

- **3.1 Converting Dictionary into Pandas Dataframe:**
 - Converting dictionary records into a unified pandas DataFrame (X).
 - Helper functions `format_rows` strips trailing newlines and whitespace.
 - `format_labels` maps integer targets (0–3) to human-readable names.
- **3.2 Familiarizing yourself with the Data:**
 - **Querying & Slicing:**
 - `loc`: Label-based indexing to select specific rows and column names.
 - `iloc`: Integer position-based indexing for zero-indexed slicing.
 - Boolean filtering: Extracting subsets with conditional masks (e.g., `X['category_name'] == 'sci.med'`).

3. Data Transformation

Q1 Data Filtering & Domain Relevance

- **Task:**
 - In Section 3.2, filter the dataframe **X** exclusively for the category **sci.med** and check its length.
- **Questions:**
 - How many documents are in this specific category?
 - From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

```
# Question 1: Filter for required category and report how many documents there are inside  
sci_med = X.loc[X['category_name'] == '?'] # Run with the correct name of the category  
len(sci_med)
```

4. Data Mining using Pandas

- **4.1 Dealing with Missing Values:**

- Checking completeness across rows and columns using `.isnull().sum()`
- Strategies: Drop rows, impute values, ignore, or predict with a model.
- Synthetic test: Injecting dummy records to verify detection and row removal with `.dropna()`

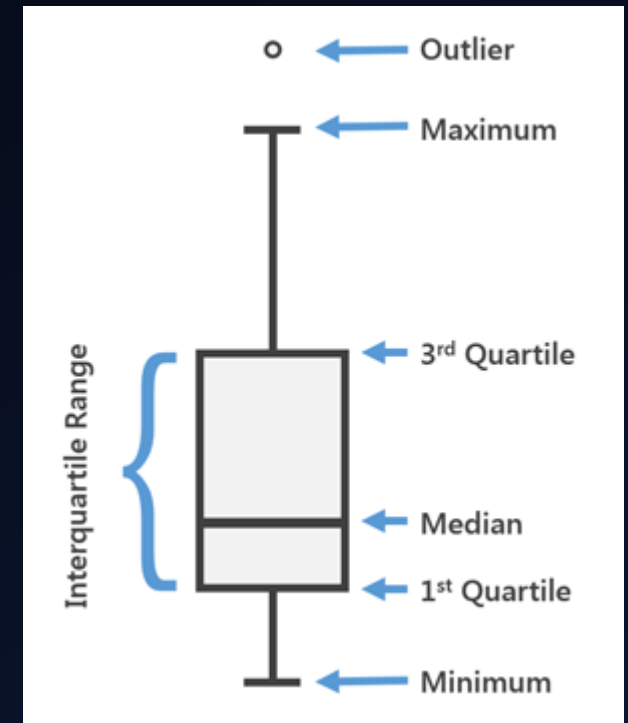
- **4.2 Dealing with Duplicate Data**

- Duplicate records artificially inflate word frequencies and distort support counts.
- Checking whole rows via `.duplicated()` vs. checking specific columns like `.duplicated('text')`
- Removing duplicates using `.drop_duplicates(keep=False)`

4. Data Mining using Pandas

4.3 Basic Statistical Summaries:

- **Feature Engineering:**
 - Creating a simple initial feature: `text_length = len(text)`.
- **Statistical Insights:**
 - `.describe()` reveals central tendency and spread: count, mean, standard deviation, min, quartiles, max.
- **Detecting Long Tails:**
 - Large gap between the 75th percentile (3rd Quartile) and maximum length indicates heavy right-skewed outliers.
 - Outlier documents can dominate vocabulary matrices and distort downstream modeling.
- **Visualizing with Boxplots:**
 - Category-level boxplots reveal varying document length profiles across topics.



4. Data Mining using Pandas

Q2 Handling Missing data

- **Task:**
 - In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.
- **Question:**
 - If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative method to handle these records so the text data can still be utilized.

5. Data Preprocessing

- **5.0 Introduction to Vectors and Matrices**
 - **Converting Words to Numbers:**
 - Documents become numeric row vectors in a high-dimensional vocabulary space.
 - Columns represent distinct words; values indicate word occurrence or count.
 - **Document-Term Matrix (DTM):**
 - Rows = Documents, Columns = Unique vocabulary terms.
 - **The Reality of Sparsity:**
 - Most documents use only a tiny fraction of the global vocabulary.
 - The resulting matrix consists almost entirely of zeros (often >98% sparse).
 - Requires memory-efficient sparse matrix representations (scipy.sparse).

	d_1	d_2	d_3	d_4	d_5
<i>romeo</i>	1	0	1	0	0
<i>juliet</i>	1	1	0	0	0
<i>happy</i>	0	1	0	0	0
<i>dagger</i>	0	1	1	0	0
<i>live</i>	0	0	0	1	0
<i>die</i>	0	0	1	1	0
<i>free</i>	0	0	0	1	0
<i>new-hampshire</i>	0	0	0	1	1

5. Data Preprocessing

- **5.1 Sampling**

- **Purpose of Sampling:**

- Reduces compute time and memory footprint during prototyping while preserving distributional properties.

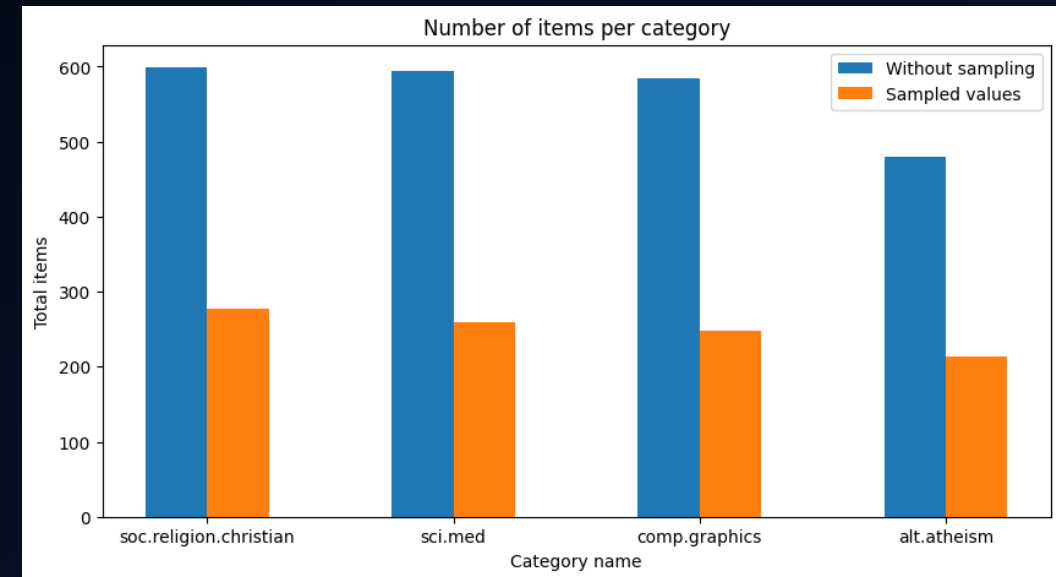
- **Using DataFrame.sample:**

- Extracting a subset of 1,000 documents using `random_state=1` for strict reproducibility.

- **The Reality of Sparsity:**

- Comparing class category frequencies between full dataset (2,257 rows) and sample (1,000 rows).
 - Visualizing side-by-side bar charts to inspect whether category proportions stay balanced.

Plot from our notebook after sampling our data



5. Data Preprocessing

Q3 Data Sampling Risks

- **Task:**
 - In Section 5.1, the code uses `X_sample = X.sample(n=1000)` to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.
- **Questions:**
 - Do the exact category proportions perfectly match the original?
 - What is the specific risk of simple random sampling on a highly imbalanced dataset?

5. Data Preprocessing

- **5.2 Feature Creation**

- **Tokenization Fundamentals:**

- Splitting continuous text strings into individual linguistic units (words, punctuation, symbols).
 - Creates an explicit unigrams list column in the DataFrame.

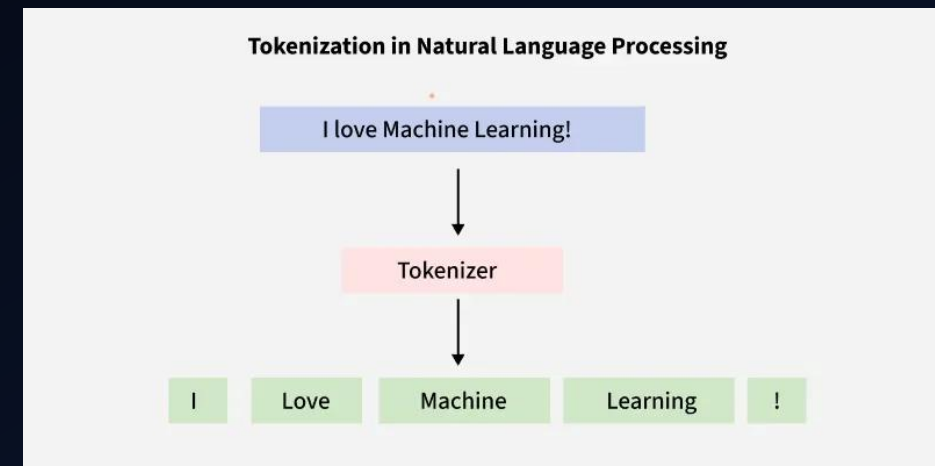
- **Using NLTK:**

- `nltk.word_tokenize` and `sent_tokenize` handle English punctuation boundaries and contractions cleanly.

- **Helper Implementation:**

- See `helpers/data_mining_helpers.py` -> `tokenize_text`.
 - Tokenized word lists provide the basis for vocabulary inspection and custom frequency counts.

Example of tokenizing a sentence



5. Data Preprocessing

- **5.3 Feature Subset Selection**

- **Vectorizing with Scikit-Learn:**

- CountVectorizer builds the global vocabulary dictionary and transforms text into a sparse count matrix (X_counts).
 - Dimensionality: 2,257 documents yield over 35,000+ unique word columns.

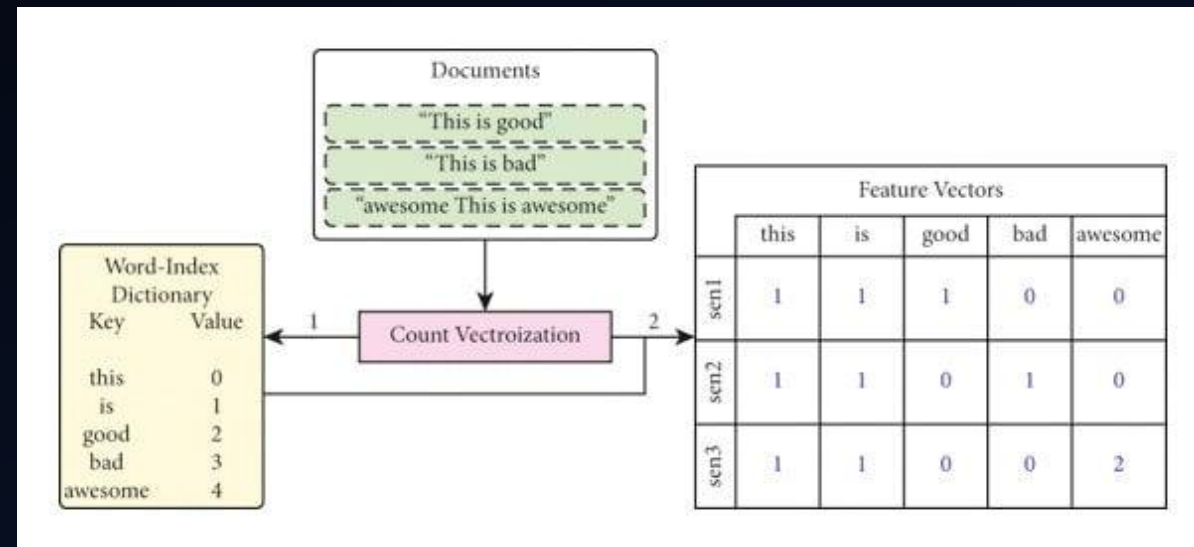
- **Stop Word Removal:**

- Parameter: stop_words="english"
 - Discards extremely common functional words (*the, is, at, which, on*).

- **Impact:**

- Focuses the feature space on informative, domain-specific terminology.

Example of CountVectorizer



5. Data Preprocessing

Q4 Text Vectorization and Matrix Sparsity

- **Task:**
 - In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.
- **Questions:**
 - Does the total number of features (columns) increase or decrease?
 - Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

```
from sklearn.feature_extraction.text import CountVectorizer
# Question 4:
count_vect = CountVectorizer(stop_words='english') # Change the parameter accordingly
# Learn the vocabulary and return document-term matrix
X_counts = count_vect.fit_transform(X.text)
print(X_counts[0])
print(X_counts.shape)
```

(np.int32(0), np.int32(28821))	1
(np.int32(0), np.int32(8638))	4
(np.int32(0), np.int32(4015))	2
(np.int32(0), np.int32(32993))	2

5. Data Preprocessing

- **5.3.1 Filtering features by variance**

- **The Variance Concept:**

- Measures column spread across all documents.
 - Terms that appear once, or terms with identical counts everywhere, exhibit near-zero variance.

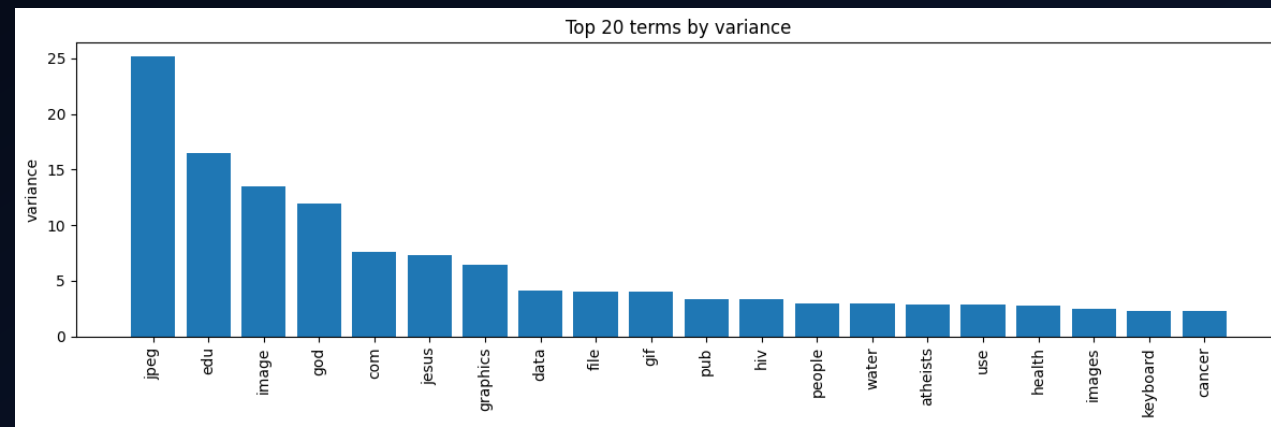
- **Unsupervised Feature Filtering:**

- Drops uninformative terms without requiring class labels.

- **Helper Implementation:**

- See helpers/feature_filtering.py -> variance_filter.
 - Scores every term by variance and sets a boolean keep flag above a chosen threshold.

Example from
our notebook



5. Data Preprocessing

Q5 Feature Selection — Variance Thresholding

- **Task:**
 - In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0**, **threshold = 0.01**, and **threshold = 0.05**. Record the number of features marked as **keep == True**.
- **Questions:**
 - What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
 - In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
 - Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

```
threshold = 0.01 # Question 5: vary this value accordingly
var_df_f = variance_filter(X_counts, feature_names, threshold=threshold)
kept = var_df_f[var_df_f["keep"]]
removed = var_df_f[~var_df_f["keep"]]

print(f"threshold = {threshold}")
print(f"kept:      {len(kept)}")
print(f"removed: {len(removed)}")
print("\nSome of the lowest-variance terms that were removed:")
removed.tail(15)
```

5. Data Preprocessing

- **5.3.2 Filtering features by correlation**

- **Supervised Feature Selection:**

- Measures the statistical association between term frequencies and class labels

- **One-vs-Rest Binary Strategy:**

- Multiclass targets become binary vectors (e.g., 1 if comp.graphics, 0 otherwise).

- **Pearson vs. Spearman:**

- **Pearson:** Measures linear correlation between term count and the class indicator.
 - **Spearman:** Measures monotonic rank correlation, resilient to extreme word-count outliers.

- **Helper Implementation:**

- See `helpers/feature_filtering.py` -> `correlation_filter_multiclass`.

5. Data Preprocessing

Q6 Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

- **Task:**
 - Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.
- **Questions:**
 - Are the top-correlated words generic stop words or domain-specific terminology for graphics?
 - What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
 - In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

```
# Question 6:
category_to_change = "sci.med" # change this value to the new category and re-run the cell.
y_cat = (X["category_name"] == category_to_change).astype(int)

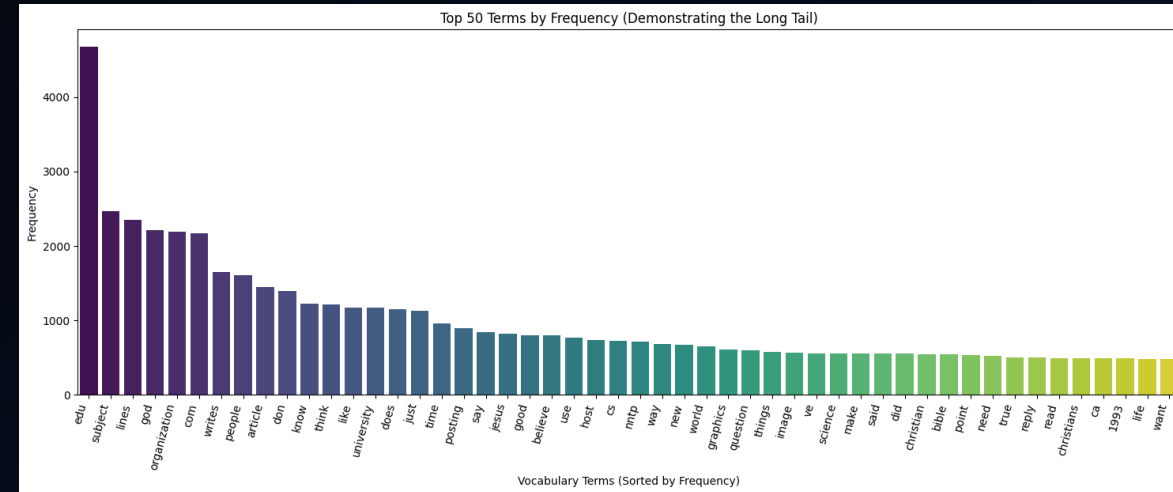
pearson_df = pearson_filter(X_counts, y_cat, feature_names)
print(f"Top terms by |Pearson r| with {category_to_change}:")
pearson_df.head(15)
```

5. Data Preprocessing

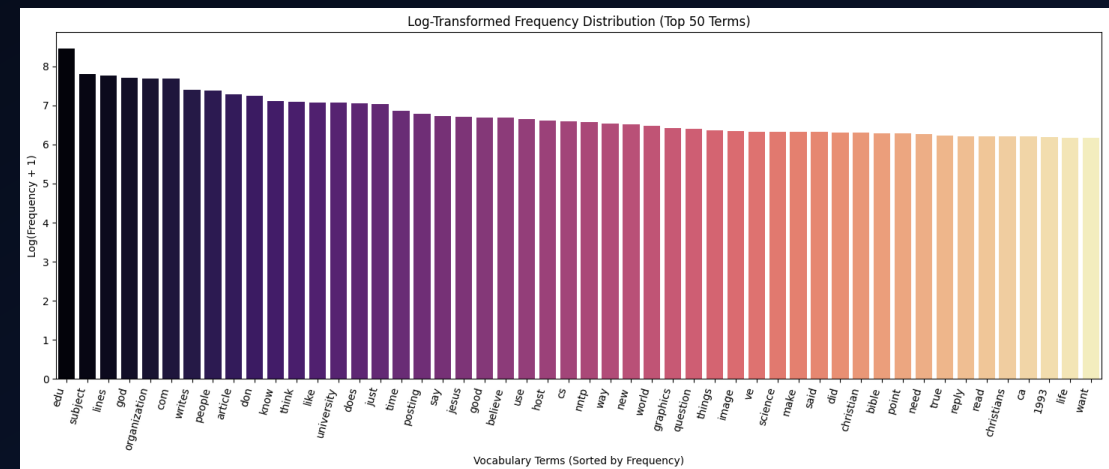
5.4 Attribute Transformation / Aggregation

- **5.4.1 Transform Text Data**
 - **Aggregating Word Counts:**
 - Summing word frequencies across all documents reveals global corpus-level term distribution.
 - **Power-Law Behavior (Zipf's Law):**
 - A handful of words occur thousands of times, while thousands of words occur only once or twice.
 - **Log-Scale Transformation:**
 - Plotting raw counts produces an unreadable vertical spike followed by a flat tail.
 - Logarithmic scaling compresses the extreme dynamic range, making the distribution interpretable.
 - **Helper Implementation:**
 - See `helpers/feature_filtering.py` -> `correlation_filter_multiclass`.

Example of the Linear Scale



Example of the Log Scale



5. Data Preprocessing

- **5.4.2 Frequent Pattern Mining**

- **Mining Co-occurrences:**

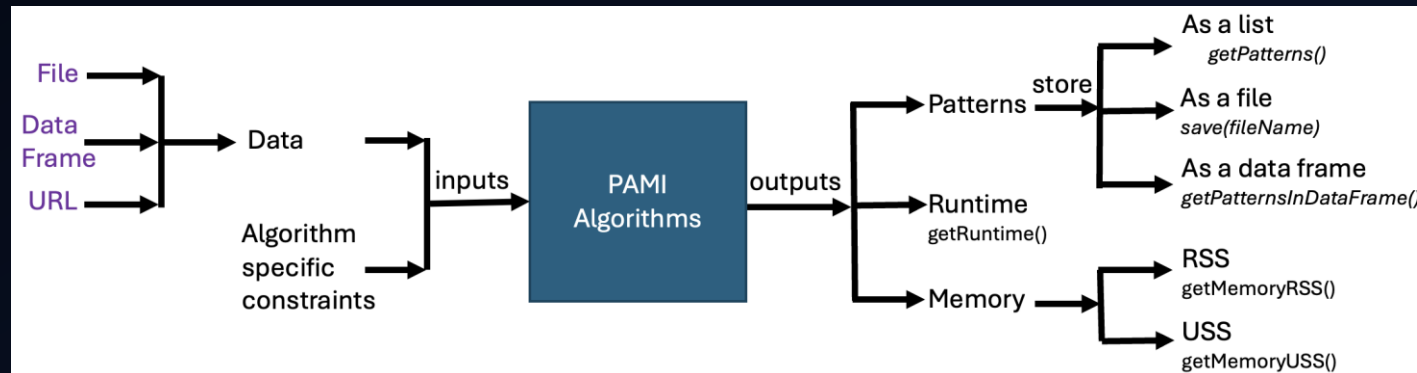
- Frequent Pattern Mining (FPM) discovers sets of words that frequently appear together within the same document.

- **Vocabulary Filtering Pipelines:**

- Raw text matrices are filtered by Variance, TF-IDF, or Term Frequency percentiles before mining (see `text_filtering_helpers.py`).

- **PAMI Library Integration:**

- Converts filtered DataFrames into transactional databases (DF2DB).
 - Runs tree-based pattern algorithms: FPGrowth, FAE and MaxFPGrowth.
 - Generates association rules evaluated by Support and Confidence



5. Data Preprocessing

Q7 Frequent Pattern Mining — Algorithm & Parameter Dynamics

- **Task:**
 - In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100** and **k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup = 9** and **minSup = 3**.
- **Questions:**
 - How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
 - How does the parameter **k** in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
 - Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

```
# Top-K FAE Algorithm: try k = 100 and k = 500
#This might take some seconds to process.
from PAMI.frequentPattern.topk import FAE as fae_alg
kCount=100 #Change this value for Question 7 between 100 and 500
category="alt_atheism" # You can change between to see what happens: alt_atheism, comp_graphics, sci_med, soc_religion_christian
# We use the term frequency processed files for this exercise
obj_k = fae_alg.FAE(iFile=f'./output_files/term_freq/transactional_dbs/td_freq_db_{category}.csv', k=kCount) #initialize
obj_k.mine() #Start the mining process
print('Runtime: ' + str(obj_k.getRuntime())) #measure the runtime

frequentPatternsDF_k = obj_k.getPatternsAsDataFrame()
print(f"Using kCount: {kCount} and extracting patterns for category: {category}")
frequentPatternsDF_k
# Run and compare runtime and pattern counts
```

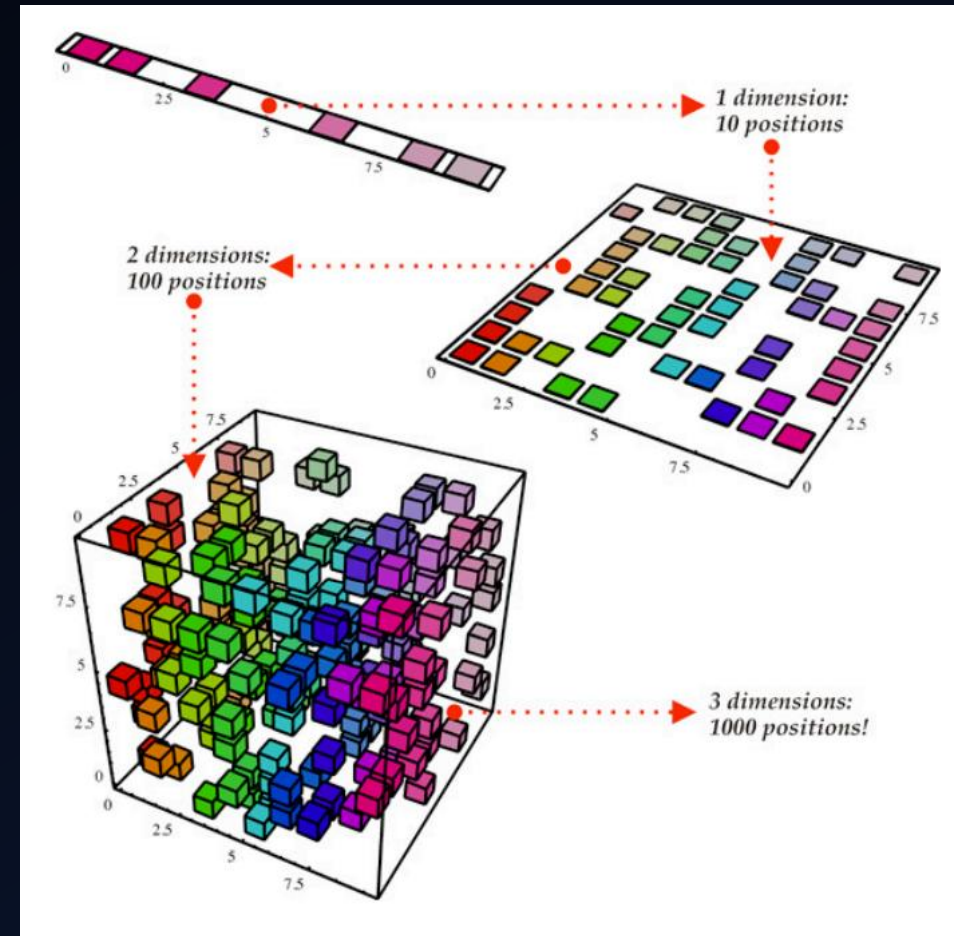
Outputs are collapsed --

```
# Maximal MaxFPGrowth Algorithm: try minSup = 9 and minSup = 3
#This might take some seconds to process.
from PAMI.frequentPattern.maximal import MaxFPGrowth as maxfp_alg
minimumSupportCount = 3 #Change this value for Question 7 between 3 and 9
# We use the term frequency processed files for this exercise
obj_max = maxfp_alg.MaxFPGrowth(iFile=f'./output_files/term_freq/transactional_dbs/td_freq_db_{category}.csv', minSup=minimumSupportCount)
obj_max.mine() #Start the mining process
frequentPatternsDF_max_comp_graphics= obj_max.getPatternsAsDataFrame()
print('Total No of patterns: ' + str(len(frequentPatternsDF_max_comp_graphics))) #print the total number of patterns
print('Runtime: ' + str(obj_max.getRuntime())) #measure the runtime

frequentPatternsDF_max = obj_max.getPatternsAsDataFrame()
print(f"Using minimumSupportCount: {minimumSupportCount} and extracting patterns for category: {category}")
frequentPatternsDF_max
```

5. Data Preprocessing

- **5.5 Dimensionality Reduction**
 - **The High-Dimensional Challenge:**
 - Our text matrix contains 35,000+ word columns. Human perception and 2D screens cannot directly plot high-dimensional spaces.
 - **Techniques Evaluated:**
 - **PCA:** Linear projection that maximizes global variance.
 - **t-SNE:** Non-linear technique preserving local neighborhood probabilities.
 - **UMAP:** Manifold learning preserving both local clusters and broader global layout.
 - **Helper Implementation:**
 - See helpers/visualization_helpers.py -> `plot_dimred_2d_comparison_grid` (creates a 3x3 comparison grid across filters and methods) and `plot_dimred_3d_multi_angle` (creates a 3x3 comparison grid across angles and methods, per filter).
 - We compare results of our original data and the augmented data result from our frequent pattern mining algorithm.



5. Data Preprocessing

Q8 Dimensionality Reduction - Motivation

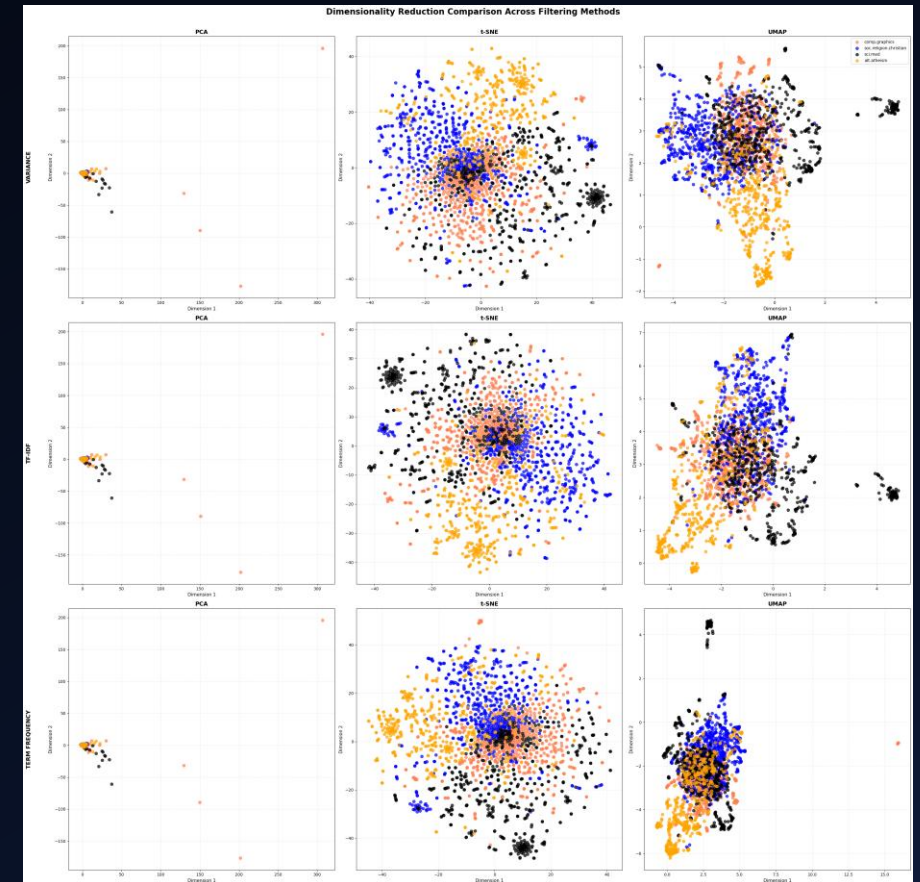
- **Task:**
 - After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).
- **Question:**
 - If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
 - What specific category of data mining algorithms (referenced in the helper files) must be applied first?

5. Data Preprocessing

Q9 Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

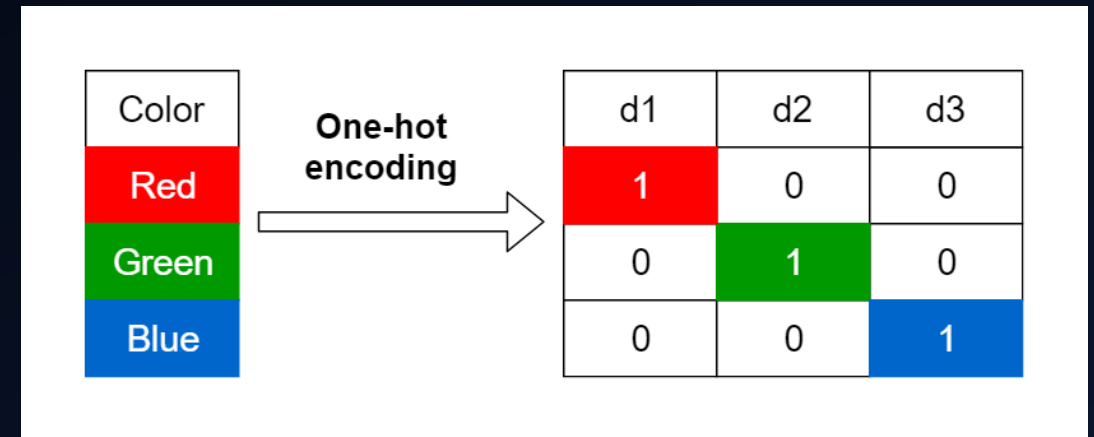
- **Task:**
 - In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.
- **Questions:**
 - Which method (**PCA** or **t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
 - Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
 - Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Example of one of the plots: 2D Plot for the Augmented Data



5. Data Preprocessing

- **5.6 Discretization & Binarization**
 - **Continuous vs. Categorical Attributes:**
 - **Discretization:** Converting continuous numbers into discrete bins (e.g., ages into age brackets).
 - **Binarization:** Transforming categorical labels into binary indicator vectors.
 - **Using LabelBinarizer:**
 - Scikit-learn's LabelBinarizer converts nominal categories into one-hot binary arrays.
 - 4 Categories become 4 orthogonal binary dimensions:
 - alt.atheism -> [1, 0, 0, 0]
 - comp.graphics -> [0, 1, 0, 0]
 - sci.med -> [0, 0, 1, 0]
 - soc.religion.christian -> [0, 0, 0, 1]



5. Data Preprocessing

Q10 Target Encoding — Integer vs. One-Hot Binarization

- **Task:**
 - In Section 3.1, examine `X['category']` (integers 0, 1, 2, 3). Then look at the output of `LabelBinarizer` in Section 5.6, where each category is converted into a binary vector (e.g., `[1, 0, 0, 0]`).
- **Questions:**
 - How many binary columns are created to represent the 4 categories?
 - If an algorithm (like linear regression or KNN) is given the raw integer column (**0, 1, 2, 3**) as input, what false mathematical assumption does it make about the categories?
 - How does one-hot binarization eliminate this false assumption?

```
# Explore binarization (see LabelBinarizer output above)
# Question 10: Uncomment one line at the time,
# and check the output to answer the questions

# X['category']
# mlb.transform(X['category']).tolist()
```

6. Data Exploration

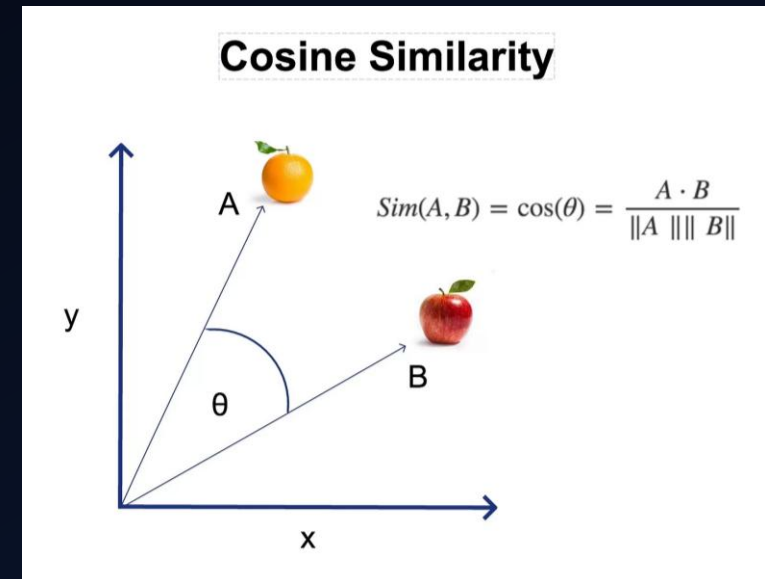
- **Document Similarity with Cosine Distance**

- **Measuring Text Similarity:**

- Comparing documents as vectors in count space.
 - Uses cosine_similarity to measure the angle between word profiles.

- **Geometric Intuition:**

- Focuses on word distribution direction rather than document length.
 - Score ranges from 0 (completely disjoint vocabulary) to 1 (identical word distribution).



Recommendations

- **Execution Workflow & Submission**

- **Notebook Execution Best Practices:**

- Run cells sequentially from top to bottom.
 - Certain heavy computation cells (like 3D manifold embeddings) are pre-saved in **output_files/**. You can enable `USE_SAVED_DATAFRAMES` to load cached results instantly rather than re-computing them

- **Submission Deliverable:**

- Complete your answers in DM2026-Lab1-Master-Questions.docx, export to **PDF**, and upload to NTU COOL.

- **Strict File Naming Convention:**

- Format: {your_name_used_in_NTU_COOL}_{student_id}_solved_master_questions.pdf
 - Example: 沙利葉_113065892_solved_master_questions.pdf

Recommendations

- **Academic Honesty, AI Usage & Resources**
 - **Generative AI Policy (Guide & Editor, Not Solver).**
 - **Permitted:** Asking AI to explain theoretical concepts, discuss algorithmic trade-offs, or polish your own written drafts.
 - **Prohibited:** Copying assignment questions directly into AI tools, fabricating experimental numbers, or submitting machine-generated prose wholesale
 - **Mandatory AI Disclosure Format Present in the Template**

References:

[Scikit-Learn Text Analytics Guide](#)
[Pandas Official Documentation](#)
[PAMI Frequent Pattern Mining Library](#)
[Understanding UMAP Manifold Learning](#)
[Numpy Cheat sheet](#)
[NLTK Tutorial](#)
[Matplotlib Tutorial](#)
[Seaborn Tutorial](#)
[Plotly Tutorial](#)
[Kaleido GitHub Repository](#)